

GROUP1

Part A — Circular & Doubly Linked List mastery

A1. CSLL: tail to head wrap vs manual reset

I predicated that non-circular sll is faster because it doesn't need to go back to the head when it hit the tail, where was CSLL is slower because it needs to loop to head.

Amount of run	CSLL	Non-circular linked list
Run1	200	401600
Run2	300	655200
Run3	300	660700
Run4	400	297800
Run5	400	505300

A2. CSLL deletion with vs without predecessor

With predecessor: $O(1)$ deletion. Without predecessor: $O(n)$ because you must find it first.

Amount of run	Case1(pred)	Case2(without pred)
Run1	5	150
Run2	6	1400
Run3	7	14500

A3. Rotate-k on CSLL & SLL

For small k, both perform similarly, but CSLL wins for large k since rotation is just pointer adjustment (no relinking)

k	CSLL (microsecond)	SLL (microsecond)
Small (10)	10	12
n/2	12	95
n-1	11	100

A4 – DLL vs SLL: Erase Given Node

DLL deletion is $O(1)$, as you can unlink via prev and next. SLL with predecessor is also $O(1)$, but without it become $O(n)$.

Part	type	average
A4	DLL	8
A4	SLL with pred	9
A4	SLL no pred	1500

A5 - Push/pop ends: head-only vs head + tail

The **tail pointer changes the story because** it eliminates traversal when inserting at the end, making back insertions constant-time instead of linear-time.

Structure	Push_back	Pop_front
SLL (head only)	1600	10
SLL (head + tail)	10	10
DLL (head + tail)	12	12

A6. Memory Overhead Audit

List type	Pointers per Node	Bytes per node	Notes
SLL	1	16B	Smallest, least flexible
CSLL	1	16B	Slightly more setup cost
SLL	2	24B	best for frequent middle deletions

Part B -- Real Worl use Case

B1. The singly linked list is faster because it stores only one and doubly linked list adds an extra prev that must be updated, doubly memory traffic for no benefit since we only use the front and end.

B2. Singly linked list will have constant-time because add and remove from the front does not depend on list size, but it may run slower.

The dynamic array will have higher average throughput even though it may have small memory spikes when it grows.

It would ship the dynamic array design, because it gives the best overall performance and memory efficiency for this Undo History system.